Behavior Driven Development

Develop thinking in the use cases

Óscar Barrios
Test Automation Specialist @SUSE

1. Product requirements
2. BDD as development process
3. Gherkin language
4. Living documentation
5. BDD in a team

1 Product requirements

Requirement = User Story + Acceptance Criteria

- **User Story**
  - Natural language **description of a feature**
  - Written from the perspective of an end user
  - A common description can follow that format:
    - As a [*role*] I want [*feature*], so that [*benefit*]

- **Acceptance criteria**
  - **Validates** that we accomplished that **user story**
  - A common acceptance criteria can follow that format:
    - Given [*initial context*], when [*event occurs*], then [*ensure an outcome*]

2 BDD as development process

2 BDD as development process

- BDD is designed to test an application's behavior from the end user's standpoint, whereas TDD is focused on testing smaller pieces of functionality in isolation.

- BDD involves product managers, developers, and test engineers who collaborate to come up with concrete examples of desirable functionality. There is a high level of communication before any implementation. By comparison, TDD can be done by a solo developer without any external input from product managers or stakeholders.

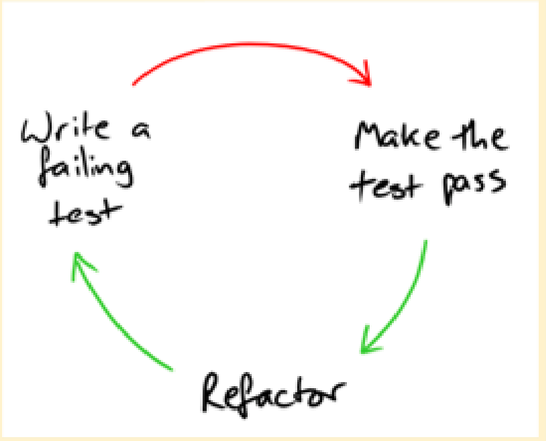
https://cucumber.io/blog/bdd/bdd-vs-tdd/

2 BDD as development process

- Was made to approach **business interests** and **technical insight**
- Using **natural language constructs** (English-like sentences), allow us to express the **behavior** and the **expected outcomes**
- Through a simple DSL (Domain-specific language) it converts natural language **statements into executable tests**

Result: Map an **acceptance criteria in a DSL** and validate the functionality at the same time that we develop it

- *Mocking* the implementation at start and iterate through it
- We will validate functionality for each change of the code



Driving design and implementation from scenarios

```
Feature: Login Profile
  As an employee of the company
  I want to login using my credentials
  So that I can collaborate with my colleagues
```

```
Background: User navigates to Company home page
  Given I am on the "Company home" page on URL "www.mycompany.com"
  Then I should see "Log In as Employee" message

Scenario: Successful login
  When I fill in "Username" with "Test"
  And I fill in "Password" with "123"
  And I click on the "Log In" button
  Then I am on the "My profile" page on URL "www.mycompany.com/myprofile"
  And I should see "Welcome to your profile" message
  And I should see the "Log out" button
```

Define User Story and its acceptance criteria

Write executable acceptance criteria

New business requirement

Acceptance tests

Design feature

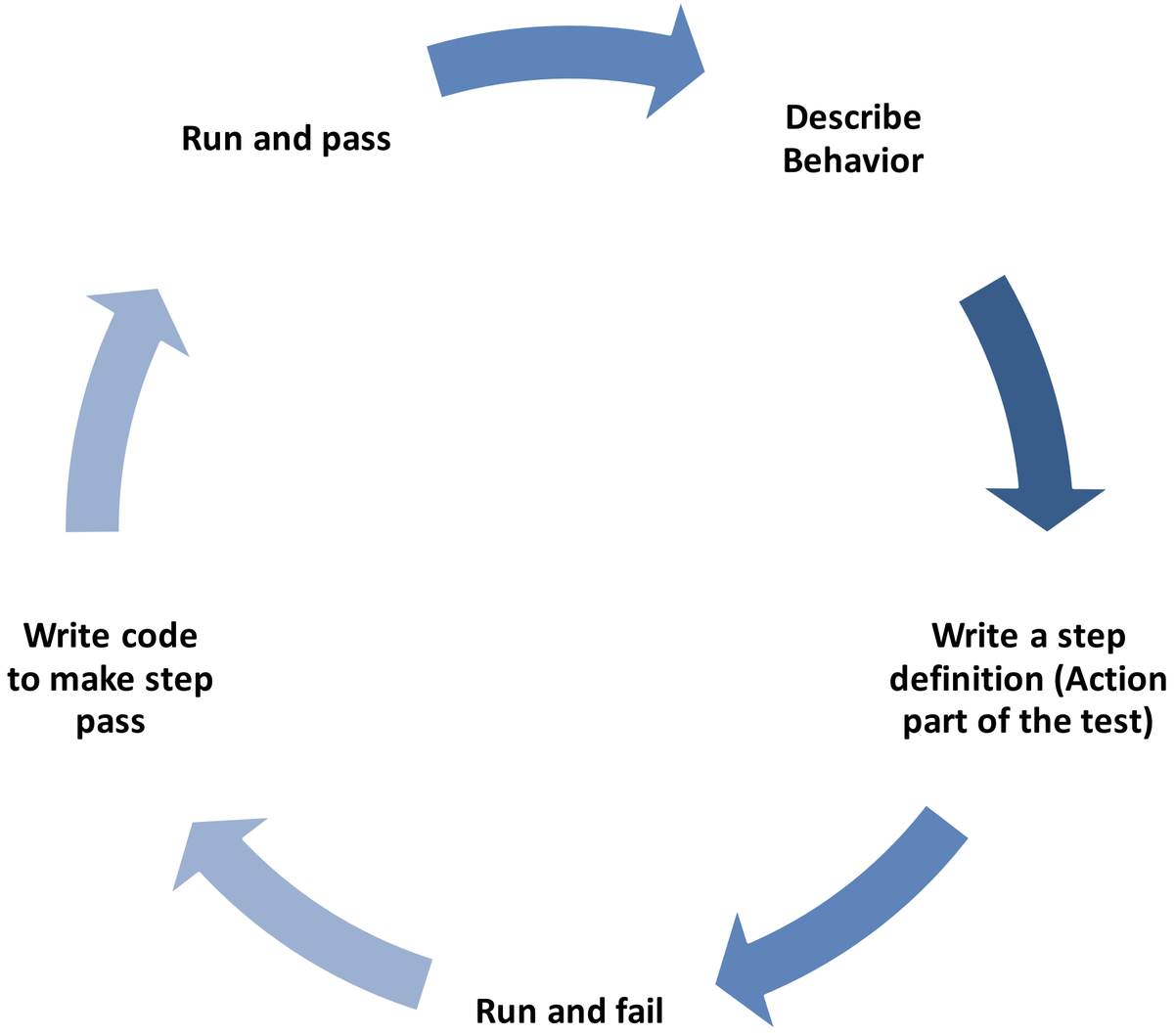
Develop feature

```
Feature: Login Profile
  Scenario: Successful login
  Scenario: Failed login using wrong credentials
```

Break feature into scenarios

Iterate through the acceptance tests to develop code

2 BDD as development process



Test after is not BDD

Many people write tests after the code is written. This is not BDD or TDD, because the tests do not *drive* the implementation when they are written afterwards.

TDD/BDD is not about testing

A common misunderstanding of TDD and BDD is that they are testing techniques. They're not. As the name suggests, TDD and BDD are about software *development*.

It is the process of approaching your design and forcing you to think about the desired outcome and API before you code.

Automated tests are a byproduct of TDD and BDD.

From https://docs.cucumber.io/bdd/overview/

3 Gherkin language

3 Gherkin language

- Is a **D**omain-**S**pecific **L**anguage business readable
- Is designed to be **non-technical** and **human readable**
- **Describes** how the system behaves
- The Gherkin language was designed for the purpose of:
  - Improving/Simplifying/Automating documentation
  - Automated tests as executable acceptance criteria

3 Gherkin language

- Uses indentation to define structure
- Has special keywords and support multiple languages:
  - *Feature*
  - *Background*
  - *Scenario*
  - *Scenario Outline*
  - *Given*
  - *When*
  - *Then*
  - *And*
  - *But*
  - *...*

Gherkin syntax - Structure

***Feature*:** Description of the user story (aka feature)

As a [*role*]
I want [*feature*]
So that [*benefit*]

***Scenario*:** Description of an acceptance criteria for that user story

Given [*initial context*]
And some other precondition
When [*event occurs*]
Then [*ensure an outcome*]
But some other postcondition will not happen

Scenario: Description of another acceptance criteria

....

Gherkin syntax - Features

- Each feature is described in a .feature file
- We will start the file with the keyword **Feature:** and its description
- The feature can contain **multiple scenarios**, where we define the acceptance criteria for that feature
- In addition to basic scenarios, a feature may contain **scenario outlines** (scenarios iterating with different inputs) and **backgrounds** (preconditions for all the scenarios)

Gherkin syntax - Scenarios

- Each scenario starts with the keyword **Scenario:**
- Describes an acceptance criteria
- A scenario **contains one or more steps**

Example:

```
Scenario: Create admin user and first organization
  Given I access the host the first time
  When I go to the home page
  And I enter "SUSE Test" as "orgName"
  And I enter "admin" as "login"
  And I enter "admin" as "password"
  And I click on "Create Organization"
  Then I am logged in
```

Gherkin syntax - Steps

- The purpose of **Given** steps is to **put the system in a known state** before the user starts interacting with the system
  - ***Given*** *the database is clean*
- The purpose of **When** steps is to **describe the key action** the user performs
  - ***When*** *I press "login" button*
- The purpose of **Then** steps is to **observe outcomes**. The observations should inspect the output of the system from a user perspective (UI, Report, command output, ...)
  - ***Then*** *the output should be "Login Successful"*
- If you have several Given, When or Then steps, you can use the step **And** or **But**, so the scenario can be read more fluently

Gherkin syntax - Background

- Allows you to **add a context to all the scenarios** of the current feature
- Is run before each of your scenarios

Example:

```
Background:
    Given I am authorized as "admin" with password "admin"
    When I follow "Software > Channels > All" in the left menu
```

Gherkin syntax - Scenario Outline

- **Rid of copy-pasted scenarios** to use different arguments
- Provides a **template using placeholders** and an “Examples” section
- The scenario it will be executed for each line of the example section

Example:

```
Scenario Outline: Eating
  Given there are <start> cucumbers
  When I eat <eat> cucumbers
  Then I should have <left> cucumbers

  Examples:
    | start | eat | left |
    |  12   |  5  |  7   |
    |  20   |  5  |  15  |
```

Gherkin syntax - Scenario Outline

```
Scenario: Create an activation key with a channel and a package list for x86_64
  Given I am on the Systems page
  When I follow "Activation Keys" in the left menu
  And I follow "Create Key"
  And I enter "SUSE Test PKG Key x86_64" as "description"
  And I enter "SUSE-PKG-x86_64" as "key"
  And I enter "20" as "usageLimit"
  And I select "Test-Channel-x86_64" from "selectedBaseChannel"
  And I click on "Create Activation Key"
  And I follow "Packages"
  And I enter "man" as "packages"
  And I click on "Update Activation Key"
  Then I should see a "Activation key SUSE Tes
  And I should see a "Details" link
  And I should see a "Packages" link
  And I should see a "Configuration" link in t
  And I should see a "Groups" link
  And I should see a "Activated Systems" link
```

```
Scenario: Create an activation key with a channel and a package list for i586
  Given I am on the Systems page
  When I follow "Activation Keys" in the left menu
  And I follow "Create Key"
  And I enter "SUSE Test PKG Key i586" as "description"
  And I enter "SUSE-PKG-i586" as "key"
  And I enter "20" as "usageLimit"
  And I select "Test-Channel-i586" from "selectedBaseChannel"
  And I click on "Create Activation Key"
  And I follow "Packages"
  And I enter "man" as "packages"
  And I click on "Update Activation Key"
  Then I should see a "Activation key SUSE Test PKG Key i586 has been modified." text
  And I should see a "Details" link
  And I should see a "Packages" link
  And I should see a "Configuration" link in the content area
  And I should see a "Groups" link
  And I should see a "Activated Systems" link
```

Gherkin syntax - Scenario Outline

```
Scenario Outline: Create an activation key with a channel and a package list for <architecture>
  Given I am on the Systems page
  When I follow "Activation Keys" in the left menu
  And I follow "Create Key"
  And I enter "SUSE Test PKG Key <arch>" as "description"
  And I enter "SUSE-PKG-<architecture>" as "key"
  And I enter "20" as "usageLimit"
  And I select "Test-Channel-<architecture>" from "selectedBaseChannel"
  And I click on "Create Activation Key"
  And I follow "Packages"
  And I enter "man" as "packages"
  And I click on "Update Activation Key"
  Then I should see a "Activation key SUSE Test PKG Key <architecture> has been modified." text
  And I should see a "Details" link
  And I should see a "Packages" link
  And I should see a "Configuration" link in the content area
  And I should see a "Groups" link
  And I should see a "Activated Systems" link
  Examples:
    | architecture |
    |  x86_64      |
    |  i586        |
```

The Gherkin syntax

- Allows multiple arguments
  - *When I write "**Jhon**" in "**Name**" text field*

- To better organize, it allows *tags* and you can filter by them

```
@billing
Feature: Verify billing
  Scenario: Missing product description
```

The Gherkin syntax

- Accepts **tables** to specify larger data set

```
Scenario:
  Given the following people exist:
    | name  | email            | phone |
    | Aslak | aslak@email.com | 123   |
    | Joe   | joe@email.com    | 234   |
    | Bryan | bryan@email.org | 456   |
```

- Accepts **multiline strings** (pystrings)

```
Scenario:
  Given a blog post named "Random" with:
    """
    Some Title
    ===============
    Here is the first paragraph of my blog post.
    """
```

Automate tests with a BDD Framework

cucumber

Ruby, Java, JavaScript, Kotlin

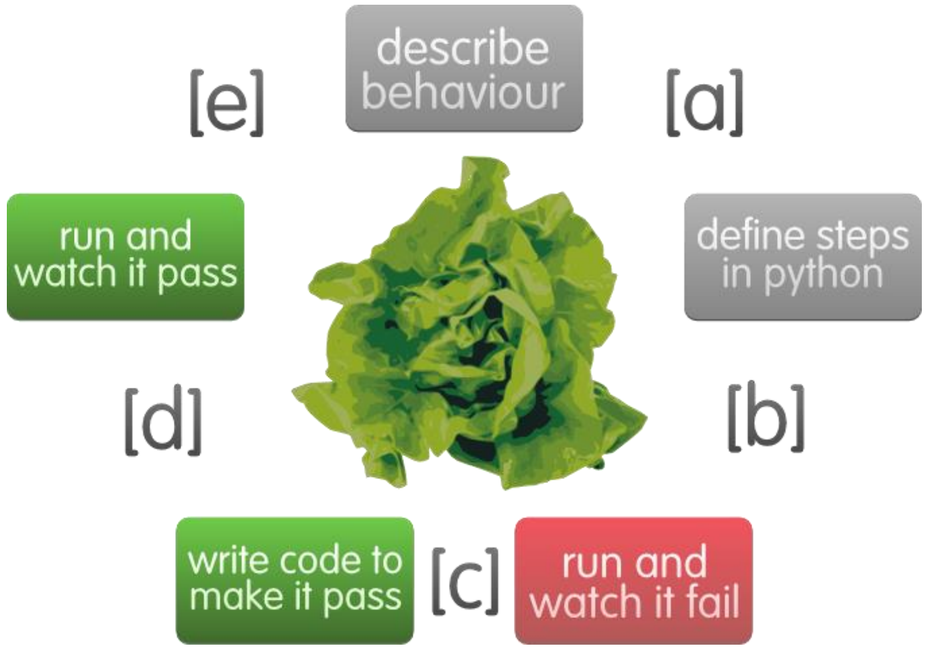
jbehave

Java

Jasmine

JavaScript

And a lot more frameworks!



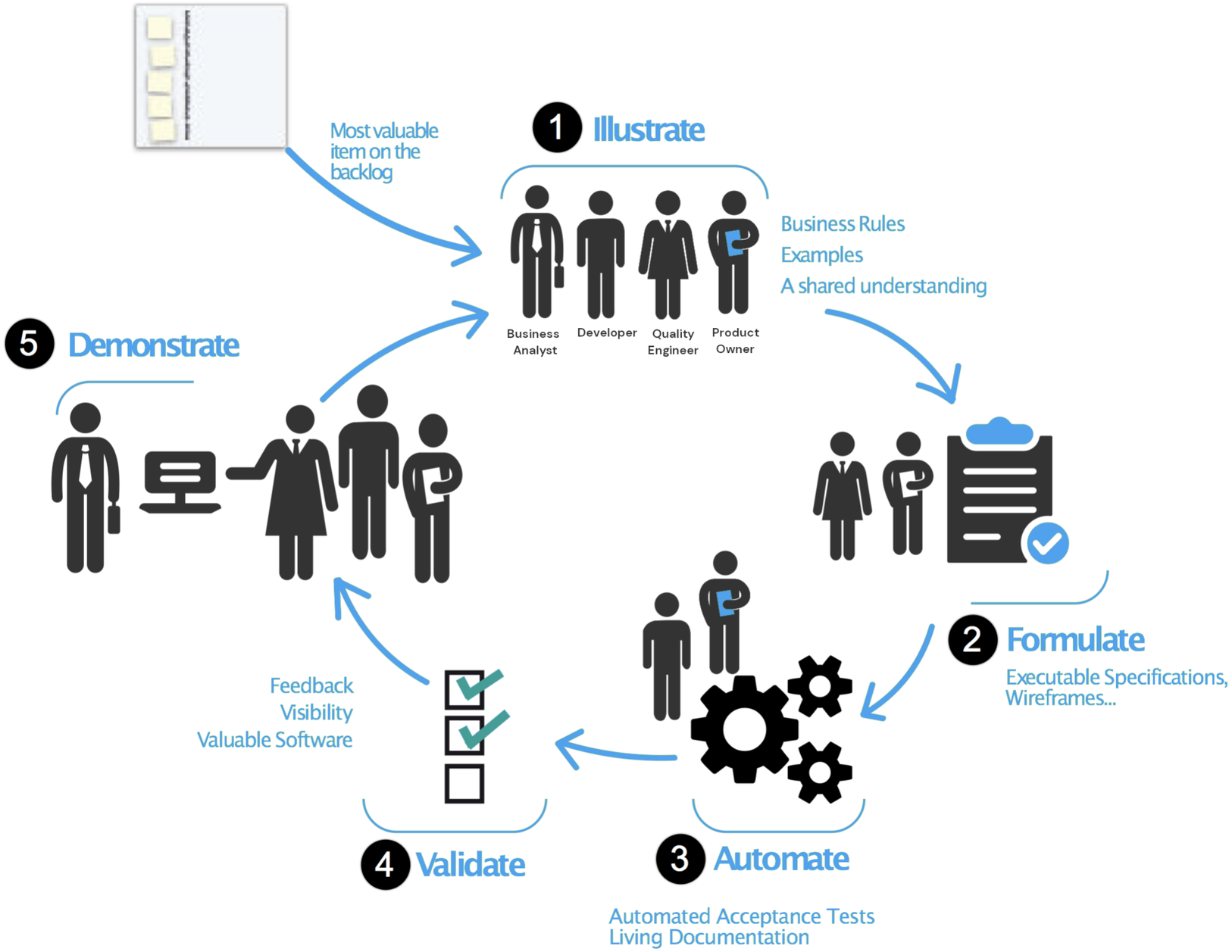
Lettuce

Python

4 Living documentation

4 Living documentation

- Living Documentation is a way to share the definition of your features and have a common support dialogue for your business and technical team members
- Writing automated tests in BDD for every feature developed or modified developers are generating documentation in natural language.
- Developers can spend their time working on new features rather than writing documentation, while stakeholders can be confident that they're always looking at the latest features.
- There are many frameworks that support living documentation generating HTML or PDF pages.

5 BDD in a team

5 BDD in a team